Dynamic Pressure Validation for Potential iOS Kernel MTE Allocator-State Races

Kaya Ercihan

24.06.2026

Abstract

This report documents the dynamic follow-up of the iOS kernel MTE tag-generation ordering hypothesis on a physical **iPhone 17e / iOS 26.5.1**. The target helper is `irg_mte_function`, recovered as `vm_memtag_generate_and_store_tag()` at `0xFFFFFE000A8578A4`. The static basis is an instruction-level ordering question: the helper performs `IRG` followed by allocation-tag stores such as `STG`, `ST2G`, and `STGM`, while the helper body itself does not contain a general-purpose `DMB/DSB/ISB` publication barrier [1, 2, 3]. The dynamic harness therefore stresses reachable userland surfaces that can drive tag-enabled kernel allocator paths, primarily Mach IPC / `ipc_port` zone allocation and teardown. The implemented execution classes are FAST, PIPELINE, FLOOD, `PREFILL_FLOOD`, `PENDING_FLOOD`, and a `BGProcessingTask` wrapper. The captured results show strong intermediate evidence: the trigger modes execute, pending ports and queued messages are created, large alloc/free counts are reached, alloc/free phases overlap, background scheduling is admitted and completed by iOS, and kernel/system logs can be used to distinguish scheduler noise from fault evidence. No run produced an observed MTE tag-fault signature. The conclusion is therefore positive for reachability and pressure generation, but negative for a confirmed same-object stale-tag race or tag-fault manifestation.

1 Scope and evidence model

The analyzed runtime target is **iPhone 17e / iOS 26.5.1**. The helper address, caller-path interpretation, and Skywalk serialization assessment are carried forward from the static caller-path and dynamic analysis notes [1, 2, 4]. The tested application source revision is the `MTETrigger.app` C/SwiftUI harness, and the runtime evidence consists of app logs, Console logs, kernel rows, system scheduler rows, screenshots, and a screen recording from the physical device [5, 6, 7].

The evidence model distinguishes intermediate runtime evidence from final fault evidence. The experiments produced several positive intermediate results: the target helper and allocator paths were identified statically; the app implemented multiple execution classes for Mach-port and message-pressure generation; the in-app logs showed pending-port reservation, queued-message creation, worker execution, alloc/free phase overlap, statistics collection, and clean teardown; and system logs showed that a background-processing run was admitted, launched, granted a background assertion, and completed.

These observations support different claims. App logs and screenshots demonstrate that the selected trigger modes executed and generated allocator pressure. Kernel and system logs provide independent evidence for scheduler behavior, runtime admission, background launch, assertion handling, and completion. However, the captured logs did not show a positive MTE-fault signature such as `TAG_FAULT`, `tagcheck`, `DFSC0x11`, an MTE-derived `EXC_BAD_ACCESS`, or a kernel panic attributable to tag mismatch. Therefore the dynamic result is not “no evidence”, it is evidence for execution of the mapped userland trigger routes, pressure generation, and background execution, but not evidence for runtime helper execution, a confirmed same-object stale-tag race, or an MTE tag fault.

Accordingly, the implemented execution classes are treated as validation stages. Foreground FLOOD validates high-rate Mach-port churn. `PREFILL_FLOOD` validates allocator-state perturbation through old live ports plus short burst ports. `PENDING_FLOOD` validates queued-message and live-port pressure before the FLOOD phase. The `BGProcessingTask` wrapper validates that the same pressure logic can be admitted and completed under iOS background scheduling. None of these execution classes is reported as a tag-fault trigger unless the corresponding Console or kernel logs contain a real MTE-fault signature.

Table 1: Concrete target and observable criteria

Field	Value
Device and OS	**iPhone 17e / iOS 26.5.1**, physical device
Kernel helper	`irg_mte_function`, recovered as `vm_memtag_generate_and_store_tag()`
Helper address	`0xFFFFFE000A8578A4`
Primary runtime surface	Mach IPC / `ipc_port` zone allocation and teardown via receive-right allocation, receive-right destruction, and optional queued Mach messages
Secondary surface	BSD socket / Skywalk slab path retained for reachability context but excluded from final TOCTOU pressure after the slab-lock serialization assessment
Positive fault criteria	`TAG_FAULT`, tag-check fault text, `DFSC0x11`, MTE-derived `EXC_BAD_ACCESS`, or MTE-derived panic
Negative fault criterion	Pressure and overlap without any tag-fault signature, without same-object kernel trace, and without allocator consistency failure

2 Static basis and dynamic target window

The target hypothesis is not that `IRG` or `STG` is incorrect as an instruction. It is an ordering question at the boundary where a newly tagged or re-tagged kernel object is published, recycled, or consumed by a different execution context. Prior static analysis classified the helper and direct caller contexts by scanning for explicit barriers and atomic/lock-like instructions. The helper and downstream tag-store worker contain MTE operations but not explicit `DMB/DSB/ISB` fences in their bodies [1, 2].

```
FFFFFE000A8578A4: BTI   c
FFFFFE000A8578A8: IRG   X0, X0, X2
FFFFFE000A8578AC: AND   X9, X0, #0xFFFFFFFFFFFFFFF0
FFFFFE000A8578B0: ADD   X8, X1, X0
FFFFFE000A8578B8: AND   X8, X10, #0xFFFFFFFFFFFFFFF0
...
FFFFFE000A8578D4: STG   X9, [X9],#0x10
FFFFFE000A8578FC: STGM  X11, [X9]
FFFFFE000A857934: ST2G  X9, [X9],#0x20
FFFFFE000A857978: ST2G  X9, [X9],#0x20
FFFFFE000A857988: ST2G  X9, [X9],#0x20
FFFFFE000A857974: RET
```

Listing 1: Reduced helper primitive: logical tag generation followed by tag stores

From stock userland the app cannot call Listing 1 directly. The mapped route to the helper traverses a Mach or BSD syscall, IPC/socket handling, reference-count updates, allocator selection, zone or slab policy checks, and the MTE-enabled allocation/free path. Therefore the app controls transition rate and lifetime pressure, not the exact instruction boundary at which `IRG` and `STG/ST2G/STGM` execute. The relevant dynamic opportunity is the same-object interval after a tag store is issued and before a tag-dependent allocation-return path, especially `vm_memtag_load_tag()`, consumes the object in a concurrent execution context, ideally on another core. Its absolute duration was not measured on the A19 implementation; it is treated as an instruction-scale, plausibly nanosecond-class interval, while the userland route includes syscall, IPC, allocator, policy, and lock-path latency [4].

Table 2: Why phase overlap is weaker than a same-object MTE race

Layer	Interpretation
Target window	Tag-store visibility after `STG/ST2G/STGM` versus later `vm_memtag_load_tag()` or tagged access on the same object.
Userland control	The app can increase allocation/free pressure and bias CPU placement, but it cannot select the exact zone element or the exact helper instruction boundary.
Overlap counter	`g_overlap_count` records phase-entry observations in which a worker sampled the opposite phase as active. It is an approximate userland workload-concurrency indicator; it does not measure overlap duration, unique overlap intervals, same-element reuse, or helper-level overlap.
Valid positive result	Requires a real tag-fault signature or kernel trace linking one element address, CPU ID, helper entry/exit, free-side retag, allocation return, and tag load.

3 Kernel surface selection

3.1 Zone allocator surface

The final candidate surface is the Mach IPC / zone allocator route. The key userland-controlled object class is `ipc_port`: a kernel object representing a Mach port endpoint, allocated from the Mach IPC port zone. Receive-right allocation is driven by `mach_port_allocate()`; teardown is driven by `mach_port_mod_refs(...,MACH_PORT_RIGHT_RECEIVE,-1)`. The important property is same-zone lifetime churn in an allocator class that can reach tag-enabled zone allocation and free-side retagging paths, not direct userland choice of the kernel allocation tag.

```
#define MTE_MODE_FAST           0  /* receive-right only */
#define MTE_MODE_PIPELINE       1  /* send right + mach_msg */
#define MTE_MODE_FLOOD          2  /* single-port ping-pong */
#define MTE_MODE_PREFILL_FLOOD  3  /* live ipc_port reserve pool */
#define MTE_MODE_PENDING_FLOOD  4  /* queued messages + FLOOD */

void mte_trigger_parallel(uint32_t num_workers, uint32_t duration_ms,
    uint32_t num_ports, uint32_t mode);
void mte_set_prefill_churn(uint32_t pct);
void mte_set_pending_messages_per_port(uint32_t count);
void mte_set_realtime_enabled(bool enabled);
```

Listing 2: Application-level zone surface and mode definitions

3.2 Skywalk exclusion

The Skywalk path is retained as a reachability and noise reference, but it is not used as the final TOCTOU pressure surface. The reason is the per-cache slab mutex. The dynamic analysis models both alloc and free bodies as executing under `skm_sl_lock`; therefore same-cache Skywalk allocation/free MTE calls are serialized with respect to each other [4, 8].

```
#define SKM_SLAB_LOCK(_skm)   lck_mtx_lock(&(_skm)->skm_sl_lock)
#define SKM_SLAB_UNLOCK(_skm) lck_mtx_unlock(&(_skm)->skm_sl_lock)

int skmem_slab_alloc(struct skmem_cache *skm, ...)
{
    SKM_SLAB_LOCK(skm);
    err = skm->skm_slab_alloc(skm, oi, oim, skmflag); /* alloc_locked */
    SKM_SLAB_UNLOCK(skm);
}

void skmem_slab_free(struct skmem_cache *skm, void *buf)
{
    SKM_SLAB_LOCK(skm);
    skm->skm_slab_free(skm, buf); /* free_locked */
    SKM_SLAB_UNLOCK(skm);
}
```

Listing 3: Condensed Skywalk serialization model

4 Implemented trigger modes

Table 3: Implemented application modes and scientific purpose

Mode	Implementation	Reason for the methodology
FAST	Receive-right-only Mach port allocation/free.	Maximizes zone lifetime transitions per second and removes send-right and message overhead.
PIPELINE	Receive right, send right, Mach message, teardown.	Better semantic coverage of alloc/publish/free but lower transition rate.
FLOOD	Repeated one-port handoff through per-worker-pair SPSC item rings; 200 ms warm-up; optional affinity and real-time policy.	Increases same-class short-lived object transitions while avoiding large simultaneous batches.
`PREFILL_FLOOD`	Reserves a live pool of `ipc_port` objects, optionally churns a fraction of the reserve, then runs FLOOD.	Tests whether many pre-existing live objects change allocator state, reuse behavior, or timing during later short-lived bursts.
`PENDING_FLOOD`	Reserves ports with queued inline Mach messages, then runs FLOOD.	Tests whether ports with pending messages and associated `ipc_kmsg`/message-buffer allocations behave differently from empty receive-right-only ports.
Background wrapper	Schedules the same C trigger as a `BGProcessingTask` with a 15 s cap and real-time priority disabled.	Changes app lifecycle/scheduler context without changing allocator logic; verifies whether the pressure path can execute when launched by DAS rather than foreground UI.
Burst-pause-release design	Not a dedicated implemented mode in the inspected source revision.	Experimental follow-up: alternate flood bursts, idle pauses, slow reserve release/reallocation, and a second flood to test allocator aging and refill hysteresis.

4.1 `FLOOD` mode: repeated single-port handoff

`FLOOD` is the core high-rate mode. In the current single-port `FLOOD`/`PENDING_FLOOD` hot path, each alloc/free worker pair repeatedly transfers one Mach receive-right port through a small per-pair ring: the alloc worker creates one port and enqueues it, while the free worker dequeues and destroys it. The large reported counts are therefore aggregate counts over time and across workers, not one simultaneous allocation wave. The separate high-throughput foreground screenshot was captured with batched `num_ports=8` semantics: `4244932` alloc batches correspond to `33959456` allocated ports. This explains the apparent batch/port mismatch and separates the batched throughput result from the later single-port handoff design. The scientific purpose remains the same: maximize short-lived `ipc_port` lifetime transitions and measure alloc/free phase overlap under tight handoff pressure.

```
/* alloc worker */
mach_port_t port = MACH_PORT_NULL;
kern_return_t kr = mach_port_allocate(task,
    MACH_PORT_RIGHT_RECEIVE, &port);
if (kr != KERN_SUCCESS) {
    atomic_fetch_add(&g_alloc_failures, 1);
    continue;
}

int32_t concurrent_frees = atomic_load(&g_active_frees);
atomic_fetch_add(&g_active_allocs, 1);
if (concurrent_frees > 0)
    atomic_fetch_add(&g_overlap_count, 1);
atomic_fetch_sub(&g_active_allocs, 1);

atomic_fetch_add(&g_alloc_ports, 1);
atomic_fetch_add(&g_alloc_batches, 1);

mach_port_t last_freed = atomic_load(ctx->shared_last_freed);
if (last_freed != MACH_PORT_NULL && port == last_freed)
    atomic_fetch_add(&g_port_name_reuse_proxy, 1);

/* free worker */
atomic_store(ctx->shared_last_freed, item.port);
kr = mach_port_mod_refs(task, item.port,
    MACH_PORT_RIGHT_RECEIVE, -1);
```

Listing 4: FLOOD hot path: one port per iteration and weak port-name reuse proxy

4.2 `PREFILL_FLOOD`: live-port occupancy shaping

`PREFILL_FLOOD` keeps a reserve set of live Mach receive-right ports before the burst phase. The purpose is to test whether a high number of pre-existing `ipc_port` objects changes allocator state, reuse behavior, or timing during subsequent short-lived alloc/free bursts. It is an allocator-occupancy experiment, not evidence of tag

mismatch by itself.

```c
static void prefill_zone(uint32_t target, mach_port_t **out_ports,
    uint32_t *out_count)
{
    mach_port_t *ports = malloc(target * sizeof(mach_port_t));
    memset(ports, 0, target * sizeof(mach_port_t));

    for (uint32_t i = 0; i < target; i++) {
        kern_return_t kr = mach_port_allocate(mach_task_self(),
            MACH_PORT_RIGHT_RECEIVE, &ports[i]);
        if (kr != KERN_SUCCESS) break;
        count++;
    }
    atomic_store(&g_prefill_count, count);
    atomic_store(&g_prefill_failures, failures);
}

static void *prefill_churn_worker(void *arg)
{
    uint32_t pct = atomic_load(&g_prefill_churn_pct);
    uint32_t chunk = ctx->count * pct / 100;
    /* release and reallocate a rotating slice of the reserve */
}
```

Listing 5: Reserve pool allocation and optional reserve churn

4.3 `PENDING_FLOOD`: queued-message pressure

`PENDING_FLOOD` reserves receive-right ports, gives them send rights, queues inline Mach messages, and then runs the same FLOOD handoff. This is not pure `ipc_port` pressure: the queued messages also create `ipc_kmsg` and message-buffer pressure. That distinction is important because any observed change may come from the combined port/message state rather than from the port zone alone.

```c
static void pending_prefill_zone(uint32_t target_ports,
    uint32_t messages_per_port, mach_port_t **out_ports,
    uint32_t *out_count)
{
    for (uint32_t i = 0; i < target_ports; i++) {
        mach_port_allocate(task, MACH_PORT_RIGHT_RECEIVE, &ports[i]);
        mach_port_set_attributes(task, ports[i],
            MACH_PORT_LIMITS_INFO, (mach_port_info_t)&limits,
            MACH_PORT_LIMITS_INFO_COUNT);
        mach_port_insert_right(task, ports[i], ports[i],
            MACH_MSG_TYPE_MAKE_SEND);

        for (uint32_t m = 0; m < messages_per_port; m++) {
            pending_inline_msg_t msg = {0};
            msg.header.msgh_bits = MACH_MSGH_BITS(MACH_MSG_TYPE_COPY_SEND, 0);
            msg.header.msgh_size = sizeof(msg);
            msg.header.msgh_remote_port = ports[i];
            mach_msg(&msg.header, MACH_SEND_MSG | MACH_SEND_TIMEOUT,
                sizeof(msg), 0, MACH_PORT_NULL, 1, MACH_PORT_NULL);
        }
    }

    mte_log("[PendingPrefill] NOTE: queued messages also create "
        "ipc_kmsg / message-buffer allocation pressure, "
        "not pure ipc_port zone pressure.");
}
```

Listing 6: Pending-message prefill before the FLOOD phase

4.4 Real-time priority control

An early version allowed FLOOD workers to request `THREAD_TIME_CONSTRAINT_POLICY`. Kernel logs showed scheduler fail-safe events when workers spun at real-time priority for at least 1000000 microseconds. The implementation was therefore changed to keep real-time priority disabled by default and to force it off in the background handler.

```
kernel  scheduler: thread unknown [...] in process MTETrigger [...]
         triggered fail-safe by spinning for at least 1000000us
         at realtime priority
```

Listing 7: Kernel scheduler fail-safe observed during real-time-priority experimentation

```c
static _Atomic bool g_rt_enabled = false;

static bool try_realtime_priority(void)
{
    if (!atomic_load(&g_rt_enabled))
```

```
        return false;
    /* THREAD_TIME_CONSTRAINT_POLICY request follows only if enabled */
}

void mte_set_realtime_enabled(bool enabled)
{
    atomic_store(&g_rt_enabled, enabled);
}
```

Listing 8: Real-time priority is gated by an explicit switch

5 Background execution wrapper

The background mode is not a daemon and is not an infinite service. It submits one `BGProcessingTaskRequest`. iOS decides when to run it. In foreground, enabling the background toggle causes the UI button to schedule the request and return. The C trigger logic is executed only when the system later launches the background task.

```
func runParallel() {
    guard !isRunning else { return }
    if useBackgroundTask {
        clearLog()
        scheduleBGTask()
        return
    }
    /* otherwise run foreground trigger */
}

func scheduleBGTask() {
    let cappedDuration = min(parallelDurationMs, 15000)
    defaults.set(Int(cappedDuration), forKey: "bg_durationMs")
    let request = BGProcessingTaskRequest(identifier: Self.bgTaskID)
    request.requiresNetworkConnectivity = false
    request.requiresExternalPower = false
    request.earliestBeginDate = Date(timeIntervalSinceNow: 1)
    try BGTaskScheduler.shared.submit(request)
}
```

Listing 9: Foreground scheduling branch: submit request and return

```
static func handleBGTask(_ task: BGProcessingTask) {
    os_log("MTETrigger: [BG] handler received")
    var didExpire = false
    task.expirationHandler = {
        didExpire = true
        os_log("MTETrigger: [BG] EXPIRED, stopping trigger")
        mte_trigger_parallel_stop()
    }

    DispatchQueue.global(qos: .userInitiated).async {
        mte_set_realtime_enabled(false)
        if mode == 4 { mte_set_pending_messages_per_port(msgsPerPort) }
        else if mode == 3 { mte_set_prefill_churn(churnPct) }

        os_log("MTETrigger: [BG] START mode=%u workers=%u duration=%u ports=%u",
               mode, workers, duration, ports)
        mte_trigger_parallel(workers, duration, ports, mode)
        os_log("MTETrigger: [BG] END expired=%d", didExpire ? 1 : 0)
        task.setTaskCompleted(success: !didExpire)
    }
}
```

Listing 10: Background handler: same C trigger, real-time priority forced off

The system logs are essential because the app UI only proves request submission. Earlier scheduler evaluations were deferred because the device was active. The successful run was later admitted when the scheduler policy allowed execution.

```
17:02:31 dasd bgProcessing-com.ercihan.MTETrigger.backgroundFlood:2EAFA5:[
  {name: Device Activity Policy, policyWeight: 5.000,
   response: {33, 0.00, [{launchType == PROCESSING AND deviceActive == 1}]}}
], Decision: MNP
```

Listing 11: Earlier background evaluations were deferred by DAS device-activity policy

```
17:23:56 dasd Submitted: bgProcessing-com.ercihan.MTETrigger.backgroundFlood:B2DF89
         at priority 10 (Wed Jun 24 17:31:26 2026 - Wed Jul 1 17:31:26 2026)
17:31:33 dasd 'bgProcessing-com.ercihan.MTETrigger.backgroundFlood:B2DF89'
         CurrentScore: 0.862875, ThresholdScore: 0.860234 DecisionToRun:1
17:31:33 dasd [FBSSystemService] Sending request to open "com.ercihan.MTETrigger"
17:31:33 SpringBoard Received trusted open application request for "com.ercihan.MTETrigger"
17:31:33 runningboardd Acquiring assertion ... "com.apple.das.backgroundTasks"
         ... BGProcessingTask-Restricted ... target:741
17:31:49 dasd COMPLETED <_DASActivity:
```

```
10          "bgProcessing-com.ercihan.MTETrigger.backgroundFlood:B2DF89", Background,
11          300s, Started at 24.06.26, 17:31:33, PID: 741> with runtime limit -1
12 17:31:49 runningboardd Invalidating assertion ... target:741
```

Listing 12: Successful background scheduling and completion for run B2DF89

6 Runtime results

6.1 Counter interpretation

The numeric counters measure the behavior of the trigger harness, not the existence of an MTE violation.

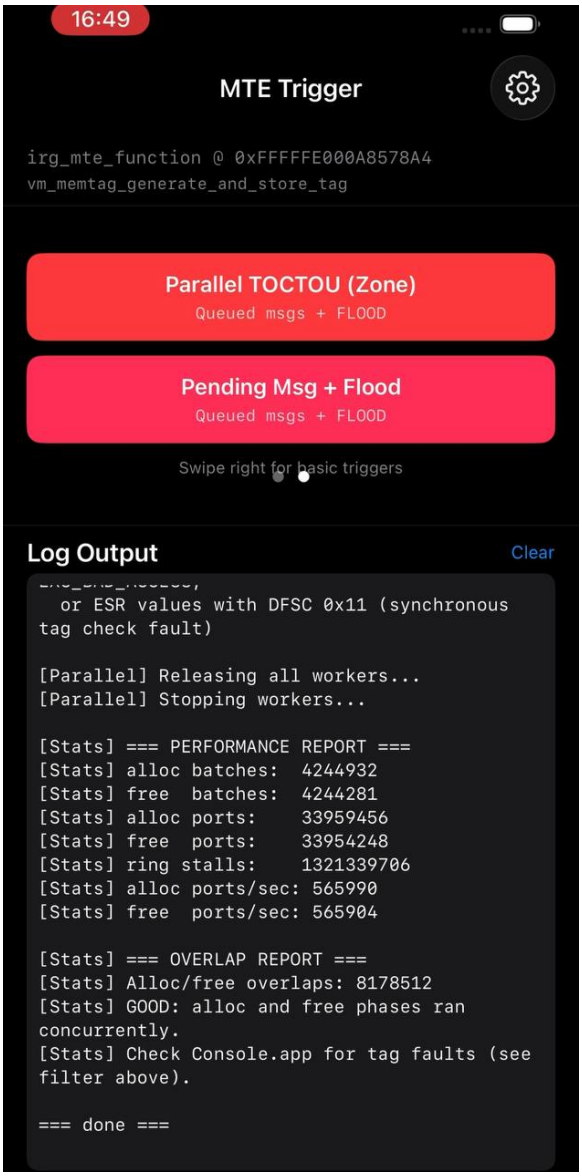
Table 4: Meaning of the main runtime counters

Counter	Interpretation
`allocbatches` / `freebatches`	Number of successful alloc/free loop iterations. Current single-port paths use one port per batch. Batched or `num_ports>1` paths use multiple ports per batch; the high-throughput run used `num_ports=8`, so `4244932` batches correspond to `33959456` ports.
`allocports` / `freeports`	Number of Mach receive-right ports allocated or destroyed by the measured workers.
`ports/sec`	Throughput normalized by the configured duration. It is the transition rate into the kernel path, not a direct helper-call rate.
`ringstalls`	Number of times an alloc worker could not enqueue immediately because its SPSC handoff ring was full. High values indicate tight handoff pressure and/or a slower consumer.
`Alloc/freeoverlaps`	Number of phase entries that observed the opposite phase as active. Provides a userland concurrency indicator; it does not establish same-element instruction overlap.
`port-namereuseproxy`	Weak proxy: a port name equal to the last freed name in a worker pair. It is not proof that the same kernel object address was reused.
`pendingfailures` / alloc/free failures	Setup or teardown error counters. Zero values mean the test ran cleanly, not that a tag fault occurred.

6.2 Observed runs

Table 5: Observed dynamic runs and interpretation

Run / evidence	Mode and setup	Observed counters	Interpretation
Early zone-only dynamic run	Parallel zone pressure on **iPhone 17e / iOS 26.5.1**	12443103 alloc/free phase overlaps; no visible tag-fault signature.	Demonstrates a high rate of opposite-phase-active observations; it does not prove same-object tag-store/load overlap [4].
Foreground high-throughput run	6 worker pairs, 60 s foreground pressure with batched `num_ports=8` semantics	4244932 alloc batches; 4244281 free batches; 33959456 alloc ports; 33954248 free ports; 565990 alloc ports/s; 8178512 overlaps.	Highest observed throughput in the provided screenshots. The batch/port ratio is expected because each batch contains up to 8 ports; clean negative tag-fault result.
Console `PENDING_FLOOD` run	1000 pending ports, 1000 queued messages, 6 worker pairs, 60 s	2051196 alloc ports; 2051186 free ports; 34186 ports/s; ring stalls 2017873841; port-name reuse proxy 5; pending failures 0; 1651622 overlaps.	Confirms pending-message prefill and clean teardown; port-name reuse proxy is nonzero but weak; no tag-fault evidence.
Demo `PENDING_FLOOD` run	1000 pending ports, 1000 queued messages, 6 worker pairs, 60 s	1950500 alloc ports; 1950506 free ports; 32508 ports/s; ring stalls 1438950263; port-name reuse proxy 5; pending failures 0; 3592 overlaps.	Shows that queued-message pressure can drastically change throughput and overlap shape; still no MTE fault.
Background `PENDING_FLOOD` run	Same pressure class through `BGProcessingTask`; duration capped to 15 s	DAS admitted and completed the task from 17:31:33 to 17:31:49; app-side screenshot series shows 279493 alloc ports, 279501 free ports, 18632 ports/s, pending failures 0, 158445 overlaps.	Proves background scheduling/execution and clean run completion; scheduler completion is not fault evidence.
RT-priority experiment	FLOOD workers with real-time policy request	Kernel scheduler fail-safe after spinning for at least 1000000 us at realtime priority.	Led to RT disabled by default and forced off in background; a scheduler artifact, not MTE evidence.



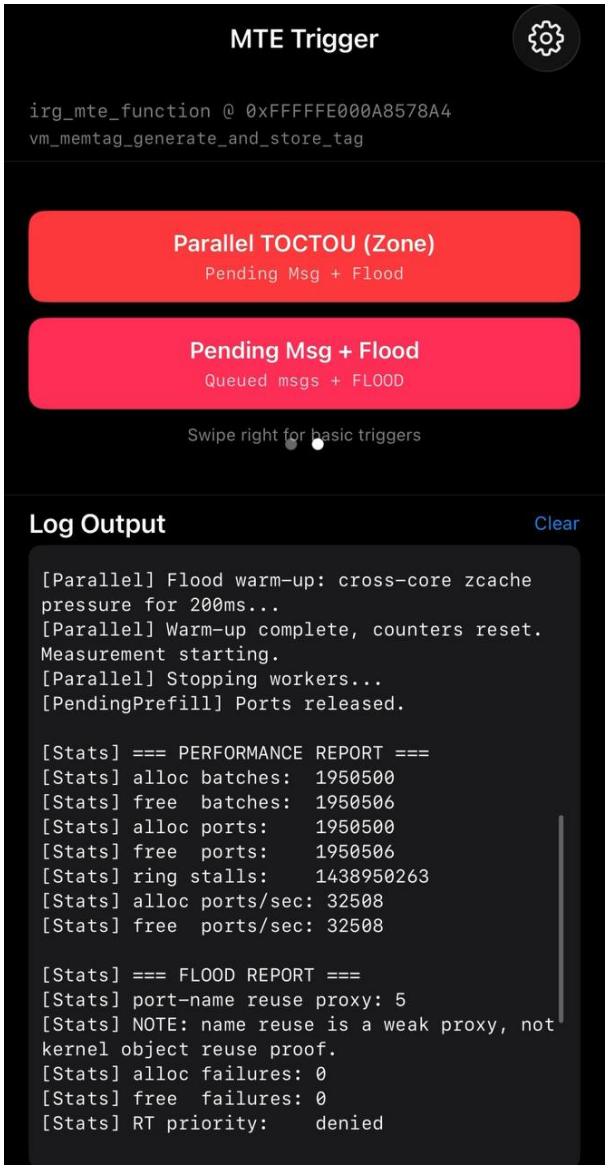


Figure 1: Foreground evidence. Left: high-throughput batched zone pressure with 33959456 allocated ports and 8178512 overlaps. Right: pending-message FLOOD after warm-up, showing port-name reuse proxy and zero alloc/free failures.

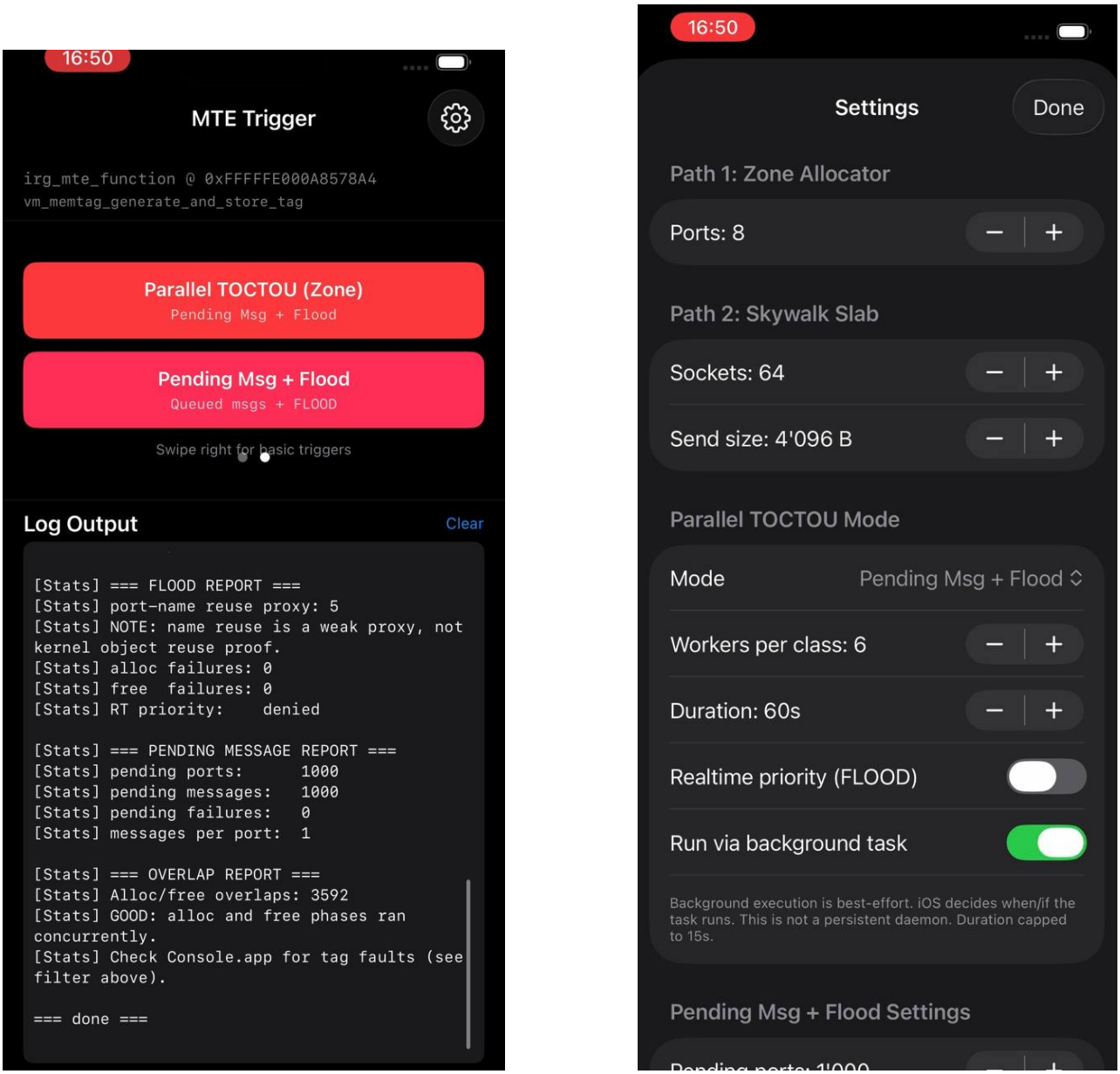


Figure 2: Pending-message result and active background configuration. The run shows 1000 pending ports, 1000 pending messages, no pending failures, and no MTE fault. The settings view shows background-task execution enabled, duration 60 s with background cap, six workers per class, and real-time priority disabled.

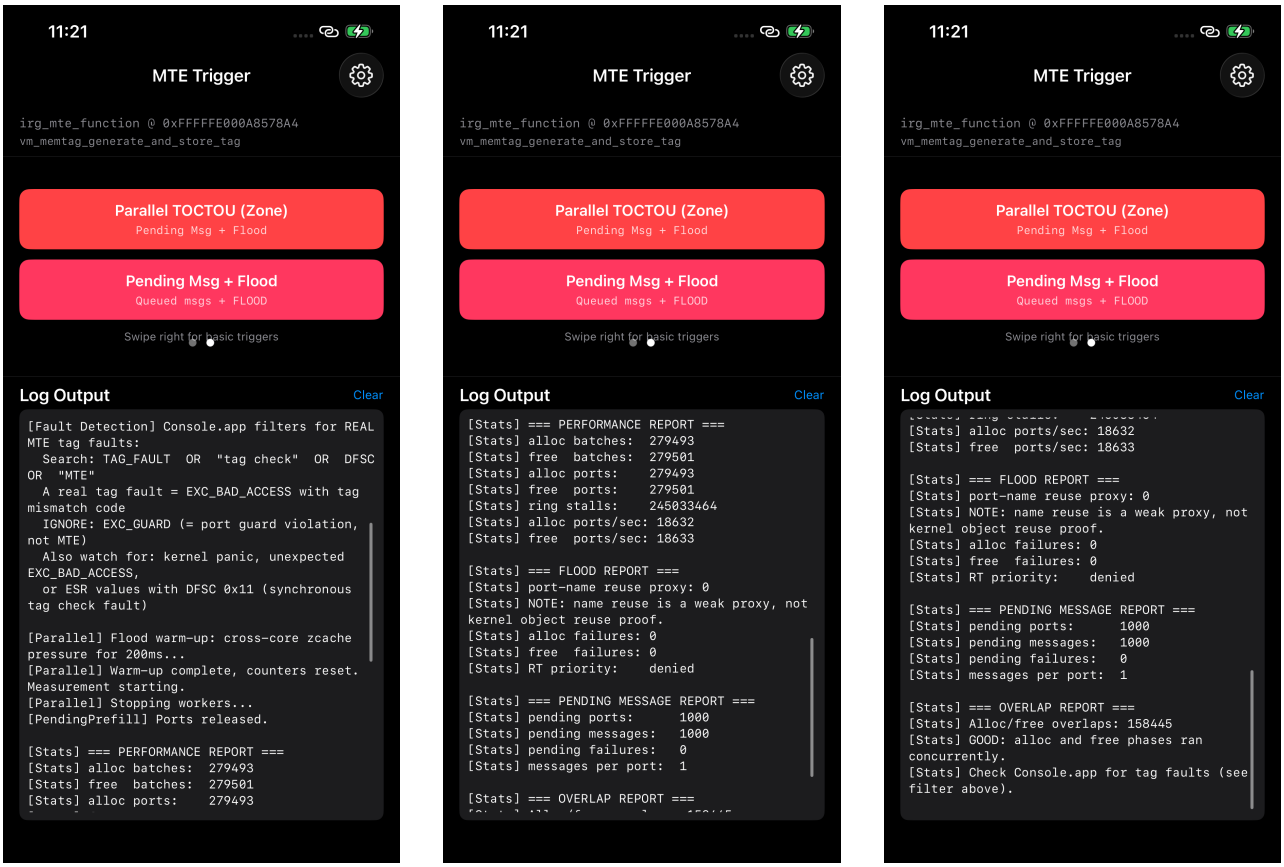


Figure 3: Background-mode app-side statistics series. The screenshots show pending-message setup, 15 s capped measurement, 279493 allocation batches, 279501 free batches, zero setup failures, and 158445 alloc/free overlaps.

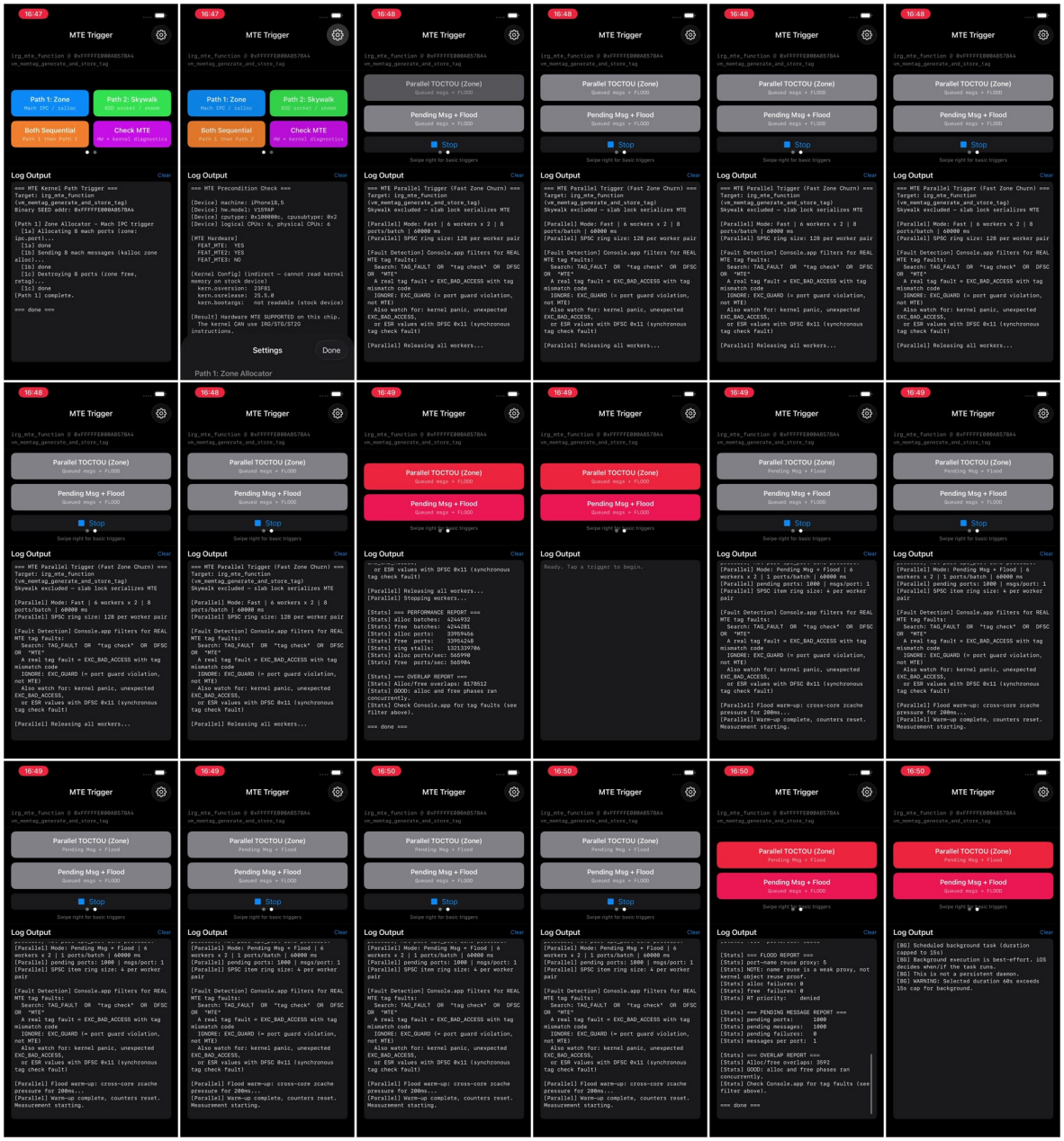


Figure 4: Representative frames extracted from the demonstration recording. The sequence shows baseline trigger execution, settings/precondition checks, high-throughput foreground pressure, pending-message FLOOD, and background task scheduling.

6.3 Fault triage result

The relevant positive signatures were explicitly searched in app output and Console/kernel captures. The strings `TAG_FAULT`, `tagcheck`, `DFSC0x11`, `EXC_BAD_ACCESS`, and panic-related wording appear in the app's own filter guide and in analysis notes, but no captured run contains a corresponding kernel or process fault line attributable to a tag mismatch. The observed kernel/system artifacts were scheduler fail-safe messages, DAS deferrals/admissions, RunningBoard assertions, SpringBoard background-launch rows, socket namespace noise such as `ADDRINUSE`, and unrelated memorystatus/thermal rows.

```
[Fault Detection] Console.app filters for REAL MTE tag faults:
  Search: TAG_FAULT OR "tag check" OR DFSC OR "MTE"
  A real tag fault = EXC_BAD_ACCESS with tag mismatch code
  IGNORE: EXC_GUARD (= port guard violation, not MTE)
  Also watch for: kernel panic, unexpected EXC_BAD_ACCESS,
  or ESR values with DFSC 0x11 (synchronous tag check fault)
```

Listing 13: Fault guide printed by the app. This is not itself a fault

```
_netns_reserve_common MTETrigger(...) port 51715: ADDRINUSE:
BSD wildcard with non-wild.
```

```
bind: Can't assign requested address

kernel  memorystatus: killing_specific_process ... [unrelated daemon]
thermalmonitord  Failed to read cpu-avg-limiter-input-w2r
```

Listing 14: Representative non-MTE noise observed during runs

As in the preceding physical-device study, no known-positive MTE fault was generated to calibrate detector sensitivity. The negative fault classification is therefore limited to the captured observation channels and does not establish absence of an unobserved tag-check event.

7 Interpretation

The experiments demonstrate that the application can drive the intended pressure classes. FLOOD produces millions to tens of millions of short-lived Mach port lifetime transitions. `PENDING_FLOOD` adds a distinct allocator-state hypothesis by keeping queued messages attached to live receive rights before the burst. `PREFILL_FLOOD` adds the live-object occupancy hypothesis by holding many old ports before short-lived churn. The background wrapper confirms that iOS can launch and complete the same trigger in a background execution context, after earlier scheduler deferrals when the device was active.

The experiments do not demonstrate exploitability. The missing observation is not "more overlap" in the abstract; it is same-object instruction-window evidence. A conclusive positive result would require either a real MTE fault or kernel-side tracing that correlates the same element through free-side retag, freelist transition, allocation return, and `vm_memtag_load_tag()`. Without that correlation, phase overlap remains pressure evidence.

Table 6: Evidence-supported conclusions

Claim	Status
The static helper and relevant allocator paths were identified.	Supported by static caller-path and dynamic notes.
The app implements multiple allocator-pressure classes.	Supported by source revision and UI/settings evidence.
Foreground batched Mach-port pressure can generate very high churn.	Supported by the 33.9M port-count screenshot and overlap counters; this run used batched `num_ports=8` semantics.
Pending-message prefill executes and tears down cleanly.	Supported by app and Console counters: 1000 pending ports, 1000 messages, zero pending failures.
Background task scheduling works.	Supported by DAS `DecisionToRun:1`, SpringBoard open request, RunningBoard assertion, and DAS completion for B2DF89.
A real MTE tag fault occurred.	Not supported by captured logs.
A same-object stale-tag race was proven.	Not supported without kernel-side element tracing or fault manifestation.

8 Recommended next validation

The preferred next validation is kernel-side correlation on an Apple SRD, debug kernel, or equivalent controlled instrumentation environment. The minimum useful trace tuple is: zone name, element address, CPU ID, helper entry/exit timestamp, free-side retag timestamp, freelist transition, allocation return, later `vm_memtag_load_tag()` or tagged access, and fault outcome. Such tracing would directly test the same-object instruction-level hypothesis rather than relying on userland pressure counters.

Where such instrumentation is unavailable, the next practical step is to extend the stock-device search space by perturbing syscall-entry timing and scheduler conditions while retaining the established allocator-pressure modes. Timing variation should be applied before allocation and teardown syscalls and the realized scheduling/timing configuration recorded for each run. This can improve black-box coverage, but conclusive validation still requires either a real external MTE fault or same-object kernel-side correlation.

Acknowledgements

The author thanks Slavo Greminger for the technical input that led to the `PENDING_FLOOD` approach, in which live Mach ports are combined with queued Mach messages before entering the parallel FLOOD phase. This idea helped extend the dynamic validation strategy beyond pure port allocation/free churn by adding pending-message pressure as a separate allocator-state hypothesis.

The author also thanks Tobias Ospelt for suggesting the background-priority and background-execution methodology, as well as the prefill-based test idea. These inputs motivated the `PREFILL_FLOOD` approach, where old live ports are retained while short-lived burst ports are churned, and the `BGProcessingTask`-based methodology, where the same trigger logic is evaluated under iOS background scheduling conditions.

References

[1] Kaya Ercihan. *Investigating a Potential TOCTOU in the iOS 26.0.1 MTE Tagging Path*. Tech. rep. Report 00. 1 March 2026. Independent Security Research, Mar. 2026.

[2] Kaya Ercihan. *iOS 26.5 MTE Tagging Paths: Caller–Trigger Correlation and TOCTOU Preconditions*. Tech. rep. Report 01. 11 June 2026. Independent Security Research, June 2026.

[3] Arm Ltd. *Arm Architecture Reference Manual for A-profile Architecture*. Architecture description for MTE instructions such as `IRG`, `GMI`, `STG`, `ST2G`, `STGM`, and barrier instructions `DMB/DSB/ISB`.

[4] Kaya Ercihan. *Physical-Device Evaluation of a Potential TOCTOU in the iOS MTE Tagging Path on iPhone 17e / iOS 26.5.1*. Tech. rep. Report 02. 16 June 2026. Independent Security Research, June 2026.

[5] Kaya Ercihan. *MTETrigger iOS Application Source Revision*. SwiftUI/C trigger application; files include `mte_trigger.c`, `mte_trigger.h`, `TriggerView.swift`, `TriggerViewModel.swift`, `MTETriggerApp.swift`, and `Info.plist`. 2026.

[6] Kaya Ercihan. *Physical iPhone 17e Console Log Captures for MTETrigger*. Console captures from 23–24.06.2026 including kernel, `MTETrigger`, `dasd`, SpringBoard, and RunningBoard rows; used for scheduler, fault, and statistics triage. 2026.

[7] Kaya Ercihan. *MTETrigger Demonstration Recording on Physical iPhone 17e*. Screen recording and extracted screenshots showing foreground and background trigger modes, settings, and runtime counters. 2026.

[8] Apple Inc. *XNU allocator and Skywalk source excerpts*. Source-level reference points used for reverse mapping of zone and Skywalk allocator behavior, including zone tagging gates and Skywalk slab locking. 2026.